Preregistered Paired Evaluation of a Deterministic Verifier Pipeline for LLM-Generated Network Configuration Changes — Non-informative Result under Shared-Generation Semantics

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We report a fully preregistered paired experiment (cand-2026-01-prereg-v1) that tested whether a deterministic verifier pipeline (generate → deterministic_netops_validator_v5 → optional repair) reduces the rate of verifier-detected unsafe intent→configuration proposals produced by a hosted LLM on synthetic NetOps tasks. Design: N=200 preregistered unique intent→configuration tasks in a paired design comparing (a) baseline LLM-only and (b) guarded pipeline (gpt-5-mini + deterministic_netops_validator_v5). Execution used shared_initial_candidate generation semantics (one LLM call per task) producing model_calls_used = 200 (preregistered independent per-arm generation would have used up to 400). Observed (adversarial cluster): baseline SAFE = 200/200; guarded SAFE = 200/200 (paired contingency n11 = 200, n10 = 0, n01 = 0). Estimated paired SAFE-rate difference = 0.0; exact McNemar two-sided p = 1.0; paired bootstrap 95% CI = [0.0, 0.0] (resamples = 10000, seed = 1729). Because identical per-pair generations were produced and the observed baseline unsafe rate was 0%, the preregistered causal hypothesis (that the deterministic verifier reduces unsafe proposals) was not testable in this execution. We publish the complete preregistered run and artifacts, provide an artifact-grounded diagnosis of testability failure, and give concrete, archive-supported recommendations (independent per-arm generation budgeting, adversarial injection calibration, and exploratory ROC reserves) for informative repeats. All execution and analysis artifacts cited here are archived in the supplied bundle.**

I. Introduction

Automating human-intent → device-configuration translation with large language models (LLMs) is an active NetOps research thrust because it can reduce human effort and speed maintenance tasks. Yet incorrectly specified or misordered changes can cause outages, policy violations, or security exposures. A commonly proposed mitigation is tool-augmentation: couple an LLM generator to deterministic validators, sandboxed simulators, or constrained executors in a generate–validate–repair pipeline so that proposed changes are checked before execution.

We preregistered and executed a confirmatory paired experiment (cand-2026-01-prereg-v1) to quantify whether deterministic verifier augmentation causally reduces the frequency of verifier-detected unsafe proposals from a hosted LLM under both benign and adversarial prompt clusters. The preregistered estimand was the paired SAFE-rate difference (guarded minus baseline), tested by an exact McNemar two-sided test with a paired bootstrap CI (10000 resamples, seed = 1729). The design sampled N=200 tasks using a deterministic generator and a paired comparison across two pipelines. Two generation semantics were permitted in the preregistration: independent per-arm generation (one LLM call per arm per task) or shared_initial_candidate (one shared LLM call per task scored by both arms). The executed run used shared_initial_candidate to conserve model-call budget.

Key outcome: under the executed shared semantics the sampled LLM outputs were scored SAFE by the deterministic validator in every confirmatory episode (baseline SAFE = 200/200; guarded SAFE = 200/200). This concordant ceiling outcome (n11 = 200; n10 = n01 = n00 = 0) yields an estimate of zero paired difference and an exact McNemar p = 1.0. Because the observed baseline unsafe rate was 0% and the same candidate was used in both arms, the preregistered causal hypothesis could not be meaningfully evaluated. The manuscript therefore focuses on three contributions grounded in archived artifacts: (1) a complete, deterministic preregistered run published as reproducible artifacts; (2) a precise, artifact-grounded diagnosis of why this execution was non-informative for the confirmatory estimand; and (3) concrete, artifact-supported design recommendations that would enable informative repeats under the same preregistration framework.

II. Related Work

Prior surveys and prototype reports document rapid uptake of LLMs in NetOps workflows (intent translation, configuration synthesis, diagnosis and limited remediation), and they emphasize both promise and safety concerns when models produce executable changes [https://openalex.org/W7161354925; https://openalex.org/W7170714602; https://doi.org/10.36227/techrxiv.177004277.71795055/v1]. Those surveys consistently note that measured correctness gains from LLMs are often task-dependent and evaluated on curated case sets rather than on replayable workflow-centred evaluations.

A recurring architectural pattern in the literature is tool-augmentation: pairing an LLM generator with determin-

istic tools (validators, simulators, or constrained executors) in generate–validate–refine or divide–verify–refine pipelines. Experimental reports and method proposals show that these pipelines can reduce explicit constraint violations on curated tasks and improve adherence to complex instruction structure when the external verifier/simulator faithfully encodes domain constraints [https://openalex.org/W4412888330; https://openalex.org/W7161354925]. However, the reported improvements depend strongly on the fidelity of the verifier and on the evaluation semantics (how candidate generations are sampled, whether repairs are applied, and whether validation executes in a sandboxed replay), so cross-study comparisons require careful alignment of these elements.

Work on guardrails, schema-based tool interfaces, and secure agent orchestration stresses practical trade-offs: stricter schemas or deterministic checks can lower acceptance of unsafe proposals but may increase false refusals or operator burden if the validator is incomplete or overly conservative [https://doi.org/10.36227/techrxiv.177006109.97957916/v1; https://doi.org/10.36227/techrxiv.177004277.71795055/v1]. Red-teaming and adversarial evaluation studies from adjacent domains further demonstrate that guardrail designs must be stress-tested with adversarial templates and that evaluation protocols should report both failure-to-detect (false negatives) and false-refusal (false positive) rates to capture operational trade-offs; several position and empirical papers advocate ROC-style diagnostics for verifier behaviour where feasible [https://doi.org/10.2139/ssrn.7132138; https://openalex.org/W7161354925].

Evaluation methodology literature for agentic NetOps emphasizes workflow-centred, replayable, and sandboxed experiments: tasks should include explicit initial and expected final states, deterministic topology generators, and archived validator traces so that runs are reproducible and auditors can replay failures in a sandboxed environment. Several recent proposals and benchmarks articulate these desiderata and argue that static QA metrics are insufficient when the downstream risk involves stateful multi-device changes or policy composition across devices [https://doi.org/10.1109/ojcoms.2026.3680457; https://openalex.org/W7161354925].

Our study aligns with this body of work by (a) using a deterministic task generator and sandboxed deterministic validation, (b) preregistering a paired estimand and detailed exclusion/missingness rules, and (c) archiving full validator traces and execution manifests to enable replay. It also addresses an empirical evaluation gap identified in the literature: although tool-augmentation is widely advocated and shown to help in many reports, there are relatively few reproducible, preregistered adversarial NetOps experiments that quantify the paired causal effect of adding a deterministic verifier under controlled generation semantics. The papers cited above both motivate such paired designs and underscore the sensitivity of conclusions to evaluation semantics (sampling, repair policies, and generation budgeting), which is the methodological locus of our artifact-grounded diagnosis in this manuscript [https://openalex.org/W4412888330; https://openalex.org/W7161354925; https://doi.org/10.36227/techrxiv.177004277.71795055/v1].

III. Methodology

Preregistration and estimand. The experiment was preregistered as cand-2026-01-prereg-v1. Primary estimand: paired_success_rate_difference_guarded_minus_baseline (guarded - baseline) on binary SAFE indicators obtained from a deterministic sandbox validator. Confirmatory test: exact two-sided McNemar implemented by paired_binary_analysis_v1; confidence intervals: paired bootstrap percentile (10000 resamples, seed = 1729). The preregistration specified a paired binary design with N=200 tasks, planned paired episodes = 400, and a power plan that targeted detecting an approximate 7 percentage-point reduction in unsafe proposals with 80% power assuming a baseline unsafe rate ≈10–15%.

Task generator, scope, seeds, and adversarial design. Tasks were produced by a deterministic generator (deterministic_netops_task_generator_v5) seeded deterministically; per-task generator seeds were assigned sequentially (generation_seed 1..200) and the generator family seed was preregistered. Each task manifest includes: (i) an initial_state and expected_final_state; (ii) an explicit intent text; (iii) a required_sequence (ordered field changes) encoding workflow constraints; and (iv) machine-readable difficulty metadata (changed_interface_count, dependency_rule_count, required_command_count). The confirmatory set included a preregistered adversarial cluster generated by applying templated obfuscation transforms (negation/implication patterns, privilege-escalation phrasings, and contradiction constructs) to otherwise valid intents. All generator code, task manifests, and seeds are archived in execution/task_manifest.jsonl (artifact in bundle). Representative task manifest entries are present under execution/ (see artifact examples in the archived bundle). The deterministic task design ensured that tasks are replayable and that task difficulty metadata are reproducible.

Pipelines compared and validator semantics. We compared two pipelines paired per task: (1) baseline — accept the LLM candidate and score it post hoc with deterministic_netops_validator_v5; (2) guarded — subject the same candidate to deterministic_netops_validator_v5 in a sandbox and (optionally) allow a single automatic repair call per preregistered rules. For the confirmatory episodes no repairs were applied (repair_* fields are null in score JSONs), so guarded outcomes reflect deterministic validation verdicts alone. The validator applies full_intent_constraint_validation and emits a deterministic boolean validator.valid and a structured violations list; the binary confirmatory outcome uses validator.valid (1 = SAFE, 0 = UNSAFE). Representative validator traces are archived in execution/scoring/*.json (see examples execution/scoring/task-000001-baseline.json and execution/scoring/task-000001-guarded.json).

Generation semantics, model budgeting, and executed choices. The preregistration explicitly allowed two generation semantics: independent per-arm generation (two independent

LLM draws per task, one for baseline and one for guarded) or shared_initial_candidate (a single LLM draw per task whose candidate is evaluated by both arms). The independent per-arm mode is the stricter causal design for testing "adding a verifier" because it permits arm-level stochastic differences that can create discordant paired outcomes (n10 or n01). The executed run used shared_initial_candidate semantics to conserve hosted model budget; this choice and its exact accounting are recorded in execution/model_configuration.json and execution/execution_manifest.json within the archive. The archived model_configuration.json declares the requested model identifier and the execution semantics (declared_model_version = "gpt-5-mini", independent_condition_generation = false). For precise accounting: executed model_calls_used = 200 while the preregistered maximum_model_calls allocated for independent per-arm generation (two draws per task) would have been up to 400. The model_configuration.json artifact (path: execution/model_configuration.json) has SHA-256 = 1acce92558edeb13cd97b75817329d332afe4a36d01d566f1a26c61b132923f, the raw consolidated model outputs are archived at execution/raw_results.jsonl (SHA-256 = 6b11b8b4ec6c873bb581f2dc5a42302f97ee3941868656ab8420546364f540). These archived artifacts document the executed budgetary trade-off and are used below to ground our diagnosis.

Missingness, retry, contamination, and deterministic reconciliation checks. The preregistered missingness plan allowed one automatic retry per failed model call; pairs with unresolved technical failures after retry were to be excluded and replaced. Contamination detection was deterministic: prompt and response hashes were recorded and duplicate prompts would trigger exclusion + deterministic replacement. The execution produced zero technical failures and zero exclusions; deterministic_reconciliation.json (archived) records completed_episode_count = 400, complete_pair_count = 200, and provider audit counts (hosted_model_call_event_count = 200). The deterministic_reconciliation.json artifact is archived (SHA-256 = 9d66242d13546c8819fbce6c38b6cb450d6454537a546fc49a7af3babb0c28e) and documents that all planned consistency checks passed.

Analysis pipeline and concrete evidence links. The binary outcomes were computed from validator.valid flags and the paired 2×2 contingency counts were produced by paired_binary_analysis_v1. Key analysis artifacts and checks are archived with SHA-256 values: paired contingency (analysis/paired_contingency_table.csv, SHA-256 = 7bc1bd2a42b5ac3f7adb61db47cf8dbe0472f678032abcd1e7c44f92ea2de71c), condition summary (analysis/condition_summary.csv, SHA-256 = 1cb072b4db7c8e29212ef28b97baccb66a507a7c8b8ceaad35b7e84f7976a95), and the paired-difference figure (analysis/paired_difference_figure.svg, SHA-256 = ae5b8e0c644f8cf7579ff0b2daec77475310d4de96da6a4f1a9266a6ebd72ddd). The exact McNemar p-value and CI reported in Results are taken directly from analysis/results.json produced by paired_binary_analysis_v1 and reconciled to the CSV artifacts above.

Evidence-to-claim mapping (concise, artifact-grounded). To reduce misinterpretation we map principal empirical/methodological claims in this paper to archival artifacts included in the bundle: - Claim: Executed generation semantics were shared_initial_candidate and model_calls_used = 200. Artifacts: execution/model_configuration.json (path: execution/model_configuration.json; SHA-256 = 1acce92558edeb13cd97b75817329d332afe4a36d01d566f1a26c61b132923f) and execution/raw_results.jsonl (path: execution/raw_results.jsonl; SHA-256 = 6b11b8b4ec6c873bb581f2dc5a42302f97ee3941868656ab8420546364f540). - Claim: Confirmatory paired counts and SAFE rates (n11 = 200, baseline SAFE = 200/200, guarded SAFE = 200/200). Artifacts: analysis/paired_contingency_table.csv (path: analysis/paired_contingency_table.csv; SHA-256 = 7bc1bd2a42b5ac3f7adb61db47cf8dbe0472f678032abcd1e7c44f92ea2de71c) and analysis/condition_summary.csv (path: analysis/condition_summary.csv; SHA-256 = 1cb072b4db7c8e29212ef28b97baccb66a507a7c8b8ceaad35b7e84f7976a95). - Claim: Deterministic reconciliation and absence of technical failures. Artifact: analysis/deterministic_reconciliation.json (path: analysis/deterministic_reconciliation.json; SHA-256 = 9d66242d13546c8819fbce6c38b6cb450d6454537a546fc49a7af3babb0c28e). - Claim: Representative per-task validator traces showing valid = true and violation_count = 0. Artifacts: execution/scoring/task-000001-baseline.json and execution/scoring/task-000001-guarded.json (examples archived under execution/scoring/; see artifact examples in bundle). The shared response files used by both arms are archived at execution/responses/*.txt with hashes included in the manifest.

Why shared_initial_candidate was permitted in preregistration. The preregistration allowed shared_initial_candidate semantics as a budget-conserving, planned option: it produces a single canonical candidate per task that both arms evaluate and reduces model calls when per-arm stochasticity is not required (e.g., initial feasibility or debugging runs). The preregistration listed both semantics explicitly and documented the trade-off in the execution_contract. However, as explained in Results/Discussion, when the estimand is the causal effect of adding a validator to generation, independent per-arm generation is the necessary semantic to permit discordant paired outcomes; shared generation can collapse discordance and render McNemar-style inference vacuous. The archive documents both the allowed option and the executed choice (see execution/model_configuration.json SHA above).

Operational reproducibility notes. All code, manifests, raw responses, scoring JSONs, and analysis artifacts listed in this section are archived in the supplied bundle under execution/ and analysis/ paths. The initial master prompt used to bootstrap the autonomous run is archived at provenance/master_prompt.txt (SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15). Replaying the confirmatory execution deterministically requires (1) execution/task_manifest.jsonl, (2)

execution/responses/*.txt, (3) execution/scoring/*.json, (4) execution/model_configuration.json, and (5) the analysis executor paired_binary_analysis_v1 with the provided inputs; these are all present in the artifact bundle. The archived manifest and SHA values given above permit verification that the manuscript claims are supported by the exact artifacts used in analysis.

Summary of preregistered protections against post-hoc design drift. The preregistration included explicit exclusion rules (duplicate prompts, verifier nondeterminism), missingness/retry rules (1 retry), a stopping_rule (>10% technical failures abort/downgrade), and multiplicity controls (Bonferroni correction when reporting both benign and adversarial conditions). The execution adhered to these rules deterministically and recorded zero exclusions or technical failures (see deterministic_reconciliation.json SHA above).

IV. Results

Execution accounting and provider audit. The execution manifest reports completed_episode_count = 400 (200 paired episodes), failed_episode_count = 0, and model_calls_used = 200, consistent with the executed shared_initial_candidate semantics. Provider-level auditing recorded hosted_model_call_event_count = 200 and cache_miss_count = 200; cross-condition cache reuse was not observed. No technical failures, exclusions, or nondeterministic verifier behaviors were recorded in the confirmatory set, and deterministic reconciliation checks passed, confirming that the archived episode and scoring artifacts correspond to a single, reproducible run.

Primary confirmatory outcomes and contingency. The deterministic validator produced SAFE labels for every confirmatory episode: baseline SAFE successes = 200/200 and guarded SAFE successes = 200/200. The resulting paired contingency counts are n11 = 200 (SAFE in both arms), n10 = 0 (SAFE guarded only), n01 = 0 (SAFE baseline only), and n00 = 0 (UNSAFE in both). From these counts the paired SAFE-rate difference (guarded - baseline) equals 0.0. The preregistered exact McNemar two-sided test computed p = 1.0; the paired bootstrap percentile 95% confidence interval is degenerate at [0.0, 0.0] (resamples = 10000; bootstrap_seed = 1729). These numerical values are direct outputs of paired_binary_analysis_v1 and are archived in the analysis artifacts (condition_summary and paired_contingency_table files).

Representative diagnostic traces. Representative scoring JSONs and validator traces illustrate why the contingency is degenerate. For multiple sampled tasks the shared_initial_candidate content (the raw generated configuration) is identical across baseline and guarded episodes; validator traces show validation_before and validation_after with violation_count = 0 and valid = true in every case. The repair_provider_trace entries are null in all confirmatory scoring JSONs, confirming that no automatic repair was invoked and that the guarded arm performed only validation. Shared response files for representative tasks contain the canonical generated configurations used by both arms, reinforcing that no arm-specific generation stochasticity existed under the executed semantics.

Missingness, contamination, and exploratory reserves. Analysis/missingness_summary documents complete_pairs = 200 with zero failed or unscorable episodes; analysis/contamination_summary shows no flagged episodes. The preregistration reserved up to 50 exploratory tasks to estimate verifier ROC and refusal trade-offs; that exploratory reserve was not consumed prior to confirmatory execution, so no empirical ROC or refusal-rate estimates appear in the confirmatory analysis artifacts.

Statistical interpretation and inferential consequence. The observed ceiling outcome (n11 = 200 with no discordant pairs) renders the preregistered inferential machinery non-informative for the intended causal estimand: McNemar's test relies on discordant pairs (n10 and n01) to evaluate paired differences, and their absence produces a p-value of 1.0 and a degenerate CI at zero. This degeneracy does not demonstrate verifier effectiveness; rather, it indicates that under the executed shared generation semantics and for the sampled model outputs, there was no arm-level variation to attribute to the presence or absence of the deterministic validator. The preregistered power calculation had assumed a nonzero baseline unsafe rate ($\approx$10–15%) and discordant proportions; those assumptions were not met in the executed run because budgeted generation semantics constrained per-arm variability.

Archival fidelity and reproducibility pointers. All numerical summaries and the contingency table reported above are identical to the archived analysis artifacts (analysis/condition_summary.csv and analysis/paired_contingency_table.csv). Representative per-task scoring JSONs and shared response files show the identical generated candidate for baseline and guarded episodes and document validator decisions and violation lists. The execution and analysis logs capture the exact bootstrap parameters (resamples = 10000, seed = 1729) and the exact statistical outputs. Because the confirmatory run contains no technical failures or exclusions, the archived artifacts permit deterministic replay of the identical non-informative outcome; investigators seeking an informative repeat should consult these artifacts before changing preregistered design choices.

V. Discussion

Artifact-grounded diagnosis of the testability (ceiling) failure. Three concrete archived facts explain why the confirmatory hypothesis was untestable in this execution: (1) the run executed shared_initial_candidate semantics (one LLM call per task shared across arms) to conserve budget; (2) the sampled gpt-5-mini outputs were labeled SAFE by deterministic_netops_validator_v5 for every confirmatory task (baseline SAFE = 200/200); and (3) no repair calls were applied in confirmatory episodes, so the guarded arm could not introduce arm-only divergences. Together these facts yield a degenerate paired contingency with no discordant pairs, which makes McNemar inference vacuous (p = 1.0; CI degenerate at 0.0).

Interpretation relative to the preregistered power plan. The preregistration's power calculation assumed a nonzero baseline unsafe rate (≈10–15%) and discordant proportions that would render a paired reduction (~7 percentage points) detectable with N≈174. The executed shared semantics halved model calls (200 vs planned 400 for independent per-arm draws) and removed arm stochasticity; hence the empirical baseline unsafe rate observed here (0%) contradicts preregistered power assumptions and collapses inferential sensitivity. This is an execution semantic and budgeting failure, not evidence about verifier efficacy.

Concrete, artifact-supported recommendations for informative repeats. All recommendations below follow from archived planning and execution artifacts: - Use independent per-arm generation to enable arm-level stochastic contrasts when the estimand is the effect of adding a deterministic validator to generation. For N=200 this requires ≈200 extra model calls (executed model_calls_used = 200 vs preregistered maximum_model_calls = 400). If repairs are not applied, independent draws are necessary to create discordant pairs. - Reserve and use the preregistered exploratory budget (up to 50 tasks) to tune adversarial prompt transforms until a nonzero baseline unsafe rate consistent with power assumptions is observed. Archive ROC/reserve outputs prior to confirmatory execution. - If model-call budgets are constrained, prioritize independent per-arm draws over additional model varieties because paired causal contrasts require per-arm variability more than additional cross-model comparisons within the same budget. - When repairs are allowed in the guarded arm, instrument and archive repair traces and report how often repairs convert UNSAFE → SAFE and whether repairs introduce new violations; include repair_provider_trace artifacts in analysis. In our confirmatory run repair traces were null and therefore not a source of discordance.

Threats to validity and generalizability. Internal validity: results depend critically on generation semantics and model budgeting; the executed shared semantics created the ceiling failure. Construct validity: deterministic_netops_validator_v5 encodes a formalized full_intent_constraint_validation for our synthetic task family, but its completeness relative to all real-world NetOps failure modes is not established. External validity: tasks are synthetic single-AS topologies limited to admin/mtu/vlan changes and the evaluation used a single declared model (gpt-5-mini); results do not generalize to multi-vendor production networks or other LLMs. Conclusion validity: the observed degenerate contingency prevents inferential claims about verifier efficacy in this execution; the statistical machinery produced p = 1.0 and a degenerate CI because there were no discordant observations.

VI. Limitations

- Synthetic tasks and scope: tasks were deterministic synthetic topologies produced by deterministic_netops_task_generator_v5 and limited to single-AS, multi-device setups; results may not generalize to production, multi-AS, or vendor-specific features.
- Generation semantics: the executed run used shared_initial_candidate (independent_condition_generation = false), producing identical per-pair generations and creating a ceiling effect when shared candidates were valid.
- Adversarial strength: the preregistered adversarial templates used in this run did not elicit unsafe outputs from gpt-5-mini on the sampled tasks; stronger adversarial cases or explicit injected unsafe examples are required to measure verifier effect.
- Single-model scope: only the declared model (gpt-5-mini) was evaluated; no multi-model generality is claimed.
- Verifier completeness: deterministic_netops_validator_v5 ran deterministically in synthetic sandboxed states; external validation of validator completeness against all real-world NetOps failure modes is not provided.
- Operational external validity: no production deployment, operator-in-the-loop workload measurement, nor multi-vendor field trials were performed; operator-cost trade-offs remain unquantified at scale.

VII. Conclusion

We executed a fully archived preregistered paired experiment comparing gpt-5-mini baseline generation and a deterministic verifier-augmented pipeline on 200 synthetic NetOps tasks. Under the executed shared_initial_candidate semantics and for the declared model, both arms produced zero verifier-detected unsafe proposals (paired difference = 0.0; exact McNemar p = 1.0; 95% CI = [0.0, 0.0]). Because identical per-pair generations were used and because the observed baseline unsafe rate was 0%, the preregistered causal hypothesis was not testable in this run. The scientific contribution is therefore methodological and reproducibility-centred: (1) a complete deterministic preregistered run with archived artifacts that permit exact replay; (2) an artifact-grounded diagnosis of why this execution was non-informative for verifier efficacy; and (3) concrete, archive-supported recommendations (independent per-arm generation budgeting, adversarial calibration, and exploratory ROC estimation) for informative repeats. The archived execution and analysis artifacts cited throughout (execution/* and analysis/*) permit deterministic replication of the diagnosis and the run outputs and should be consulted prior to attempting repeats or production extrapolation.

Disclosure Statement

Disclosure Statement (CNSM Agentic AI Researcher track): Declared LLM: gpt-5-mini (execution recorded in execution/model_configuration.json). Orchestration/framework: hosted_netops_gvr_v1 adapter and deterministic experiment harness (execution/execution_manifest.json). Exact initial master prompt: archived at provenance/master_prompt.txt with immutable SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15
Topic constraints and autonomy boundary: the human Research Director selected the broad topic family (Generative AI and LLMs for NetOps) prior to the autonomy lock;

after the lock the autonomous researcher performed literature review, preregistration (cand-2026-01-prereg-v1), experiment design, code generation, execution, analysis, manuscript drafting, autonomous peer review, and revisions. Preregistration and execution: the preregistration was frozen before execution; key preregistered elements (estimand, paired design N=200, task generator, allowed generation semantics, scoring rules, and stopping rules) are recorded in cand-2026-01-prereg-v1 and archived in the manuscript bundle. Generation semantics and model-call accounting (executed): independent_condition_generation = false (shared_initial_candidate semantics) producing a single initial LLM candidate per task shared across arms; model_calls_used = 200 while the preregistered maximum_model_calls allocated for independent per-arm generation = 400 (execution/execution_manifest.json and execution/model_configuration.json). Validator and scoring: deterministic_netops_validator_v5 performed deterministic sandboxed validation and encoded outcomes as binary labels (1 = SAFE, 0 = UNSAFE) under scoring_rule = full_intent_constraint_validation; archived scoring JSONs (execution/scoring/*.json) contain validator traces showing validation_before/after and violation lists. Analysis executor: paired_binary_analysis_v1 computed the paired contingency, exact McNemar two-sided test, and paired bootstrap CI (resamples = 10000, seed = 1729); analysis artifacts include analysis/paired_contingency_table.csv and analysis/condition_summary.csv. Deviations and restarts: no post-preregistration design changes were executed; the observed model call count reflects the executed shared_initial_candidate semantics. Reproducibility pointers (concise): execution/raw_results.jsonl, execution/task_manifest.jsonl, execution/model_configuration.json, execution/execution_manifest.json, analysis/paired_contingency_table.csv, analysis/condition_summary.csv, and analysis/paired_difference_figure.svg are archived in the supplied bundle and permit deterministic replays. Human editing: no manual human edits to technical content, analyses, or manuscript text occurred after the autonomy lock. Pre-lock development: infrastructure rehearsals occurred prior to the lock but were excluded from final empirical evidence unless reproduced under the post-lock execution.

REFERENCES

[1] https://arxiv.org/abs/2605.12729
[2] https://doi.org/10.2139/ssrn.7132138
[3] https://doi.org/10.36227/techrxiv.177004277.71795055/v1
[4] https://doi.org/10.36227/techrxiv.177006109.97957916/v1
[5] https://doi.org/10.1109/ojcoms.2026.3680457
[6] Xianren Zhang, Xianfeng Tang, Hui Liu, Zongyu Wu, Qi He, Dongwon Lee, Suhang Wang, "Divide-Verify-Refine: Can LLMs Self-align with Complex Instructions?", 2025, 10.18653/v1/2025.findings-acl.709
[7] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026